```python
"""Week 3 DSP + ML demonstrations: z-transform, ROC, poles/zeros, stability."""
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from scipy.io import wavfile


def partial_sum_right_exponential(a=2.0, r=1.0, omega=0.0, N=30):
    """Partial sums of sum_{n>=0} a^n z^{-n}, z=r exp(j omega)."""
    n = np.arange(N)
    z = r*np.exp(1j*omega)
    terms = (a**n)*(z**(-n))
    return np.cumsum(terms)


def plot_convergence():
    plt.figure(figsize=(8,4))
    for r in [1.0, 2.1, 2.5, 3.0]:
        s = partial_sum_right_exponential(a=2.0, r=r, N=30)
        plt.plot(np.arange(1,len(s)+1), np.abs(s), label=f"r={r}")
    plt.yscale('log')
    plt.xlabel('N'); plt.ylabel('|partial sum|'); plt.title('ROC for 2^n u[n]: convergence requires r>2')
    plt.grid(True, alpha=.25); plt.legend(); plt.show()


def inverse_z_residues():
    # X(z) = 1 / [(1-.5 z^-1)(1-.8 z^-1)]
    b = [1.0]
    a = np.convolve([1,-0.5],[1,-0.8])
    residues, poles, direct = signal.residuez(b,a)
    print('denominator coefficients:', a)
    print('residues:', residues)
    print('poles:', poles)
    print('direct term:', direct)
    print('ROC > .8 => both residue terms are right-sided')
    print('.5 < ROC < .8 => .5-pole term right-sided, .8-pole term left-sided')


def pzplot(b, a, title='Pole-zero plot'):
    z = np.roots(b) if len(b)>1 else np.array([])
    p = np.roots(a) if len(a)>1 else np.array([])
    fig, ax = plt.subplots(figsize=(5,5))
    circle = plt.Circle((0,0),1,fill=False,linestyle='--')
    ax.add_patch(circle)
    if len(z): ax.scatter(z.real,z.imag,s=90,facecolors='none',edgecolors='C0',linewidths=2,label='zeros')
    if len(p): ax.scatter(p.real,p.imag,s=90,marker='x',linewidths=2,label='poles')
    ax.axhline(0,lw=.7); ax.axvline(0,lw=.7); ax.set_aspect('equal'); ax.grid(True,alpha=.2)
    ax.set_xlim(-1.25,1.25); ax.set_ylim(-1.25,1.25); ax.set_title(title)
    if len(z) or len(p): ax.legend()
    plt.show()


def geometric_magnitude(b, a, omega):
    """Magnitude from zero/pole distances, with coefficient normalization."""
    z = np.roots(b) if len(b)>1 else np.array([])
    p = np.roots(a) if len(a)>1 else np.array([])
    # Rewrite B(z)=b0 prod(1-zk z^-1), same magnitude factor at |z|=1 as prod|e^jw-zk|
    point = np.exp(1j*omega)
    num = abs(b[0]) * np.prod(np.abs(point-z)) if len(z) else abs(b[0])
    den = abs(a[0]) * np.prod(np.abs(point-p)) if len(p) else abs(a[0])
    return num/den


def compare_geometric_to_freqz():
```

```
    b = np.array([1.0, 1.0])        # zero at -1
    a = np.array([1.0, -0.75])      # pole at +0.75
    for omega in [0, .25*np.pi, .5*np.pi, np.pi]:
        _, H = signal.freqz(b,a,worN=[omega])
        print(f"omega/pi={omega/np.pi:.2f}: geometry={geometric_magnitude(b,a,omega)
        :.6f}, freqz={abs(H[0]):.6f}")


def notch_coefficients(f0=60.0, fs=8000.0, r=0.985):
    w0 = 2*np.pi*f0/fs
    zeros = np.exp(1j*np.array([w0,-w0]))
    poles = r*zeros
    b = np.poly(zeros).real
    a = np.poly(poles).real
    b *= np.sum(a)/np.sum(b)  # DC normalization
    return b, a


def plot_notch(f0=60.0, fs=8000.0, r=0.985):
    b,a = notch_coefficients(f0,fs,r)
    pzplot(b,a,f'{f0:g} Hz notch, r={r}')
    f,H = signal.freqz(b,a,worN=32768,fs=fs)
    plt.figure(figsize=(8,4)); plt.plot(f,20*np.log10(np.maximum(np.abs(H),1e-8)))
    plt.xlim(0,300); plt.ylim(-90,3); plt.axvline(f0,ls='--'); plt.grid(True,alpha=.25)
    plt.xlabel('Hz');plt.ylabel('magnitude (dB)');plt.title('Notch response');plt.show()


def pole_radius_sandbox(theta=.35*np.pi, radii=(.6,.85,.95,.985)):
    plt.figure(figsize=(8,4))
    for r in radii:
        poles = r*np.exp(1j*np.array([theta,-theta]))
        a=np.poly(poles).real; b=np.array([1.0])
        w,H=signal.freqz(b,a,worN=4096)
        H=H/np.max(np.abs(H))
        plt.plot(w/np.pi,20*np.log10(np.maximum(np.abs(H),1e-5)),label=f'r={r}')
    plt.ylim(-60,2);plt.xlabel('omega/pi');plt.ylabel('normalized magnitude (dB)');plt.
    grid(True,alpha=.25);plt.legend();plt.show()


def hum_demo(fs=8000, duration=3.0, f0=60.0, r=.985):
    t=np.arange(int(fs*duration))/fs
    voice=(0.35*np.sin(2*np.pi*220*t)+0.18*np.sin(2*np.pi*440*t)+0.10*np.sin(2*np.pi*660*
    t))*(0.75+0.25*np.sin(2*np.pi*2.3*t))
    hum=0.28*np.sin(2*np.pi*f0*t)+0.08*np.sin(2*np.pi*2*f0*t)
    x=voice+hum
    b,a=notch_coefficients(f0,fs,r)
    y=signal.lfilter(b,a,x)
    return t,x,y,b,a


if __name__ == '__main__':
    print('--- z-transform convergence ---')
    s1=partial_sum_right_exponential(a=2,r=1,N=12)
    s2=partial_sum_right_exponential(a=2,r=2.5,N=12)
    print('unit-circle last partial magnitude:',abs(s1[-1]))
    print('r=2.5 last partial sum:',s2[-1], 'theoretical:',1/(1-2/2.5))
    print('\n--- residues ---')
    inverse_z_residues()
    print('\n--- geometric check ---')
    compare_geometric_to_freqz()
    b,a=notch_coefficients()
    print('\n60 Hz notch coefficients:')
    print('b=',b);print('a=',a)

```